



Actividad 15% 3ª evaluación IP – 1º DAW

Sergio Ruiz Montesinos

Profesores: José Luis López, Esteban Castañer, Pedro Campos

26/04/2026

LoveCode

Contenido

1. Introducción.....	4
2. Descripción General del Sistema	4
2.1 Base de datos (MySQL)	4
2.2 Backend (Java).....	4
2.3 Frontend (HTML y CSS).....	4
3. Bases de Datos	5
3.1 Estructura de la base de datos.....	5
3.2 Usuarios de la base de datos y permisos	5
3.3 Procedimientos almacenados	6
4. Programación (Java)	8
4.1 Estructura del proyecto.....	8
4.2 Conexión a la base de datos mediante JDBC	8
5. Lenguajes de Marcas (Frontend).....	9
5.1 Página de inicio — index.html	9
5.2 Formulario de login — login.html	9
5.3 Formulario de registro — register.html.....	9
6. Entornos de Desarrollo (Git)	10
6.1 Estructura del repositorio.....	10
7. Sistemas Informáticos	10
7.1 Base de datos en máquina virtual.....	10
7.2 Backup de la base de datos	11
8. Problemas Encontrados y Soluciones	11
8.1 Configuración de acceso remoto a MySQL.....	11
8.2 Credenciales en la clase conexionbd	11
8.3 Posicionamiento CSS en la página de inicio.....	12
9. Conclusiones	12
9.1 Lo aprendido en esta fase	12
10. Anexos	13

Backend	13
Frontend	14
Base de datos	17

1. Introducción

LoveCode es una aplicación web orientada a la conexión entre personas con intereses relacionados con la programación y la tecnología. El concepto se basa en el modelo de aplicaciones de perfiles, donde los usuarios pueden descubrir a otros, dar likes y, si el interés es mutuo, generar un match.

2. Descripción General del Sistema

2.1 Base de datos (MySQL)

Se utiliza MySQL como sistema gestor de base de datos. La base de datos LoveCode contiene la tabla de usuarios, los usuarios de acceso con sus permisos diferenciados y tres procedimientos almacenados que cubren las operaciones principales.

2.2 Backend (Java)

El backend está desarrollado en Java. En esta fase se ha implementado la estructura del proyecto y la conexión a la base de datos mediante JDBC, así como una consulta básica para verificar que la conexión funciona correctamente.

2.3 Frontend (HTML y CSS)

La parte visual está desarrollada con HTML y CSS. En esta fase se han creado las páginas estáticas: página de inicio, formulario de login, formulario de registro y una vista de perfiles. No hay conexión con el backend todavía.

3. Bases de Datos

3.1 Estructura de la base de datos

La base de datos se denomina LoveCode. La tabla principal creada en esta fase es usuario, que almacena la información de cada persona registrada en la plataforma.

Campo	Tipo	Descripción
id_usuario	INT (PK, AUTO_INCREMENT)	Identificador único del usuario
nombre	VARCHAR (100)	Nombre del usuario
email	VARCHAR (150)	Correo electrónico (único)
contrasena	VARCHAR (255)	Contraseña del usuario
descripción	TEXT	Descripción del perfil
fecha_registro	DATETIME	Fecha de alta en la plataforma
ciudad	VARCHAR (100)	Ciudad del usuario
estado_cuenta	ENUM ('activo', 'inactivo')	Estado de la cuenta

3.2 Usuarios de la base de datos y permisos

Se han creado tres usuarios en MySQL con niveles de acceso diferenciados según su rol dentro del sistema:

Usuario root (administrador)

Tiene todos los privilegios sobre la base de datos LoveCode. Es el usuario administrador del sistema.

```
CREATE USER 'root'@'localhost' IDENTIFIED BY '1234';
GRANT ALL PRIVILEGES ON LoveCode. * TO 'root'@'localhost';
FLUSH PRIVILEGES;
```

Usuario desarrollador

Tiene permisos de lectura, inserción, actualización y borrado. Es el usuario que utiliza el backend Java para operar con la base de datos.

```
CREATE USER "desarrollador"@"localhost" IDENTIFIED BY "DevPass456";
GRANT SELECT, INSERT, UPDATE, DELETE, CREATE, DROP, ALTER ON
LoveCode. * TO 'desarrollador'@'localhost';
FLUSH PRIVILEGES;
```

Usuario lector

Solo tiene permisos de lectura (SELECT). Es un usuario de solo consulta, pensado para tareas de monitorización o reporting.

```
CREATE USER 'lector'@'localhost' IDENTIFIED BY 'lectorpass';
GRANT SELECT ON LoveCode. * TO 'lector'@'localhost';
FLUSH PRIVILEGES;FLUSH PRIVILEGES;
```

3.3 Procedimientos almacenados

Procedimiento 1: Conteo de matches

Devuelve el número total de matches que tiene un usuario determinado.

```
DELIMITER //

CREATE PROCEDURE conteo_matches()

BEGIN

SELECT COUNT(*) AS total_matches

FROM matches;
```

```
END //
```

```
DELIMITER //
```

Procedimiento 2: Carga de datos de usuario

Devuelve todos los datos del perfil de un usuario a partir de su identificador.

```
DELIMITER //
```

```
CREATE PROCEDURE datos_usuario (IN p_id_usuario INT)
```

```
BEGIN
```

```
SELECT nombre, email, contrasena, descripcion, fecha_registro, ciudad, estado_cuenta
```

```
FROM usuario
```

```
WHERE id_usuario = p_id_usuario;
```

```
END //
```

```
DELIMITER ;
```

Procedimiento 3: Borrado de usuario

Elimina un usuario de la base de datos a partir de su identificador.

```
DELIMITER //
```

```
CREATE PROCEDURE borrado_usuario (IN p_id_usuario INT)
```

```
BEGIN
```

```
DELETE FROM usuario WHERE id_usuario = p_id_usuario;
```

```
END //
```

```
DELIMITER ;
```

4. Programación (Java)

4.1 Estructura del proyecto

El proyecto Java está organizado con dos clases principales:

- `conexionbd.java` — Gestiona la conexión con la base de datos mediante JDBC.
- `Main.java` — Clase principal que utiliza la conexión para realizar una consulta de prueba.

4.2 Conexión a la base de datos mediante JDBC

La clase `conexionbd` centraliza los parámetros de conexión y expone el método estático `getConnection()` para obtener una conexión activa. Los datos de configuración son:

URL: `jdbc:mysql://192.168.220.129:3306/LoveCode`

Usuario: `sergio\%`

Contraseña: `1234`

```
public static Connection getConnection() {  
    Connection connection = null;  
    try {  
        connection = DriverManager.getConnection(URL, USER, PASSWORD);  
        System.out.println("Conexion exitosa a la base de datos");  
    } catch (SQLException e) {  
        System.out.println("Error de conexion: " + e.getMessage());  
    }  
    return connection;  
}
```


5. Lenguajes de Marcas (Frontend)

En esta fase el frontend es puramente estático: HTML estructurado con CSS aplicado. No hay JavaScript ni comunicación con el backend. Se han desarrollado las siguientes páginas:

5.1 Página de inicio — `index.html`

Muestra el logo de la aplicación, el nombre LoveCode y un eslogan. Incluye dos botones de navegación en la esquina superior derecha: 'Iniciar sesión' y 'Registrarse', que redirigen a `login.html` y `register.html`.

Los estilos de esta página están en `style.css`.

5.2 Formulario de login — `login.html`

Contiene un formulario de inicio de sesión con los siguientes campos:

- Campo de correo electrónico.
- Campo de contraseña.
- Checkbox de 'Recordar mi usuario'.
- Botón para entrar
- Enlace a recuperación de contraseña.
- Enlace a `register.html` para usuarios sin cuenta.
- Botón de regreso al menú principal.

Los estilos están en `login.css`.

5.3 Formulario de registro — `register.html`

Contiene un formulario de registro con los siguientes campos:

- Nombre y Apellido.
- Ciudad y Dirección.
- Nombre de usuario.
- Correo electrónico.

- Contraseña.
- Botón de registrarse
- Botón de regreso al menú principal

Los estilos están en register.css.

6. Entornos de Desarrollo (Git)

Se ha creado un repositorio Git para gestionar el control de versiones del proyecto. El repositorio contiene los tres componentes principales del sistema organizados en carpetas.

6.1 Estructura del repositorio

```
LoveCode/  
base-de-datos  
BaseDeDatos_proyecto_3evaluacion.sql  
backend/  
conexionbd.java  
Main.java  
frontend/  
index.html  
login.html  
register.html  
style.css  
login.css  
register.css
```

7. Sistemas Informáticos

7.1 Base de datos en máquina virtual

La base de datos que utilizo es mariadb está instalada y funcionando en una máquina virtual con sistema operativo Debian. La dirección IP de la máquina virtual es

192.168.220.129, que es la que se utiliza en la cadena de conexión JDBC del backend Java.

Se ha verificado que la base de datos es accesible desde la máquina anfitriona a través del puerto.

7.2 Backup de la base de datos

Se ha realizado una copia de seguridad completa de la base de datos utilizando el comando `mysqldump`.

Comando utilizado para generar el backup:

```
mysqldump -u root -p LoveCode > backup_LoveCode.sql
```

El fichero `backup_LoveCode.sql` ha sido subido al repositorio Git dentro de la carpeta `database/`, junto con el script principal de creación de la base de datos.

8. Problemas Encontrados y Soluciones

8.1 Configuración de acceso remoto a MySQL

Al intentar conectar el backend Java con la base de datos en la máquina virtual surgieron errores de acceso. El problema era que MySQL por defecto solo acepta conexiones desde `localhost`. Se solucionó configurando el usuario desarrollador con el host `'%'` para permitir conexiones desde cualquier dirección IP, y verificando que el puerto 3306 estaba abierto en el firewall de la máquina virtual.

8.2 Credenciales en la clase `conexionbd`

El nombre de usuario MySQL contenía caracteres especiales que causaban problemas al pasarlo como cadena de texto en Java. Se revisó y corrigió la definición de las constantes `USER` y `PASSWORD` en la clase `conexionbd`.

8.3 Posicionamiento CSS en la página de inicio

El uso de position: fixed en los títulos h1 y h2 de la página de inicio causaba solapamientos con otros elementos en determinados tamaños de ventana. Se ajustaron los valores de top y z-index para que los elementos no interfirieran entre sí.

9. Conclusiones

En esta primera fase del proyecto se ha construido la base sobre la que se desarrollará el resto del sistema. Se ha implementado correctamente la estructura de la base de datos con sus usuarios y procedimientos almacenados, la conexión Java-MySQL mediante JDBC, el diseño estático del frontend y el control de versiones con Git.

9.1 Lo aprendido en esta fase

- Gestión de usuarios y permisos en MySQL.
- Creación y uso de procedimientos almacenados.
- Conexión a bases de datos desde Java mediante JDBC.
- Maquetación de formularios con HTML5 y CSS.
- Uso básico de Git para control de versiones y backup.
- Configuración de MySQL en máquina virtual con acceso remoto.

10. Anexos

Backend

Código completo conexionbd.java

```

1  import java.sql.Connection;
2  import java.sql.DriverManager;
3  import java.sql.SQLException;
4
5  public class conexionbd {
6
7      private static final String URL = "jdbc:mysql://192.168.220.129:3306/LoveCode";
8      private static final String USER = "sergio\%";
9      private static final String PASSWORD = "1234";
10
11     public static Connection getConnection() {
12         Connection connection = null;
13
14         try {
15             connection = DriverManager.getConnection(URL, USER, PASSWORD);
16             System.out.println("Conexión exitosa a la base de datos");
17         } catch (SQLException e) {
18             System.out.println("Error de conexión: " + e.getMessage());
19         }
20
21         return connection;
22     }
23 }
24

```

Código completo consulta simple

```

1  import java.sql.Connection;
2  import java.sql.ResultSet;
3  import java.sql.Statement;
4
5  public class Main {
6
7      public static void main(String[] args) {
8
9          try (Connection conn = conexionbd.getConnection()) {
10
11              if (conn != null) {
12
13                  String query = "SELECT * FROM usuario";
14
15                  Statement stmt = conn.createStatement();
16                  ResultSet rs = stmt.executeQuery(query);
17
18                  while (rs.next()) {
19                      System.out.println("ID: " + rs.getInt("id_usuario"));
20                      System.out.println("Nombre: " + rs.getString("nombre"));
21                      System.out.println("Email: " + rs.getString("email"));
22                      System.out.println("Contraseña: " + rs.getString("contrasena"));
23                      System.out.println("Descripcion: " + rs.getString("descripcion"));
24                      System.out.println("Fecha registro: " + rs.getString("fecha_registro"));
25                      System.out.println("Ciudad: " + rs.getString("ciudad"));
26                      System.out.println("estado cuenta:" + rs.getString("estado_cuenta"));
27                      System.out.println("-----");
28                  }
29
30              }
31
32          } catch (Exception e) {
33              e.printStackTrace();    Print Stack Trace
34          }
35      }
36

```

Frontend

Código index.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <link rel="stylesheet" href="style.css">
7      <title>Document</title>
8  </head>
9  <body>
10     <br><br>
11     
12
13
14     <h1>Conecta con gente y comparte ideas</h1>
15     <h2>LoveCode</h2>
16
17     <div class="nav-buttons">
18         <a href="login.html">Iniciar sesion</a>
19         <a href="register.html">Registrarse</a>
20     </div>
21
22 </body>
23 </html>

```

Codigo login.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <link rel="stylesheet" href="login.css">
7      <title>Iniciar sesión</title>
8  </head>
9  <body>
10     
11     <div class="container">
12         <h2>Iniciar sesión</h2>
13
14         <label for="email">Correo electrónico</label>
15
16         <input type="email" id="email" name="email" placeholder="ejemplo@gmail.com" required>
17         <br><br>
18
19         <label for="password">Contraseña</label>
20         <input type="password" id="password" name="password" placeholder="Contraseña" required>
21         <br><br>
22
23         <div class="recordar">
24             <input type="checkbox" id="recordar" name="recordar">
25             <label for="recordar">Recordar mi usuario</label>
26         </div>
27         <br>
28         <input type="submit" value="Entrar">
29         <br><br>
30         <div class="links">
31             <a href="olvidaste tu contraseña">¿Olvidaste tu contraseña?</a>
32         </div>
33         <br>
34         <div class="links">
35             <a href="register.html">¿No tienes una cuenta? Regístrate</a>
36         <br><br>
37         </div>
38         <div class="link">
39             <button><a href="index.html">Menu</a></button>
40         </div>
41     </div>
42
43 </form>
44 </body>
45 </html>

```

Código register.html

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <link rel="stylesheet" href="register.css">
7      <title>Document</title>
8  </head>
9  <body>
10     
11     <div class="container">
12         <h1>Registro</h1>
13
14         <label>Nombre</label>
15         <input type="text" placeholder="nombre">
16
17         <label>Apellido</label>
18         <input type="text" placeholder="apellido">
19
20         <label>Ciudad</label>
21         <input type="text" placeholder="ciudad">
22
23         <label>Dirección</label>
24         <input type="text" placeholder="dirección">
25
26         <label>Usuario</label>
27         <input type="text" placeholder="usuario">
28
29         <label>Correo</label>
30         <input type="email" placeholder="ejemplo@gmail.com">
31
32         <label>Contraseña</label>
33         <input type="password" placeholder="Contraseña">
34
35         <input type="submit" value="Registrarse">
36
37         <div class="link">
38             <button><a href="index.html">Menu</a></button>
39         </div>
40     </div>
41
42 </body>
43 </html>

```

Codigo style.css para la parte de index.html

```

1  * {
2      margin: 0;
3      padding: 0;
4      box-sizing: border-box;
5  }
6
7  body {
8      font-family: Arial, sans-serif;
9      background: #ffffff;
10     display: flex;
11     flex-direction: column;
12     align-items: center;
13     padding-top: 60px;
14 }
15
16 img {
17     width: 100%;
18     max-width: 1200px;
19     height: 200px;
20     object-fit: cover;
21     border-radius: 8px;
22     margin-bottom: 28px;
23 }
24
25 h1 {
26     font-size: 22px;
27     color: #000000;
28     margin-bottom: 24px;
29     left: 160px;
30     position: fixed;
31     top: 400px;
32     display: flex;
33     flex-direction: row;
34     gap: 20px;
35     z-index: 100;
36 }
37
38 }
39
40 h2 {
41     font-size: 28px;
42     color: #000000;
43     margin-bottom: 20px;
44     text-align: center;

```

```

45     position: fixed;
46     top: 320px;
47     display: flex;
48     flex-direction: row;
49     gap: 20px;
50     z-index: 100;
51 }
52
53 a {
54     display: block;
55     width: 200px;
56     padding: 11px;
57     text-align: center;
58     border-radius: 6px;
59     font-size: 15px;
60     text-decoration: none;
61     margin-bottom: 12px;
62 }
63
64 .nav-buttons {
65     position: fixed;
66     top: 16px;
67     right: 12px;
68     display: flex;
69     flex-direction: row;
70     gap: 20px;
71     z-index: 100;
72 }
73
74 a:first-of-type {
75     background: #ffffff;
76     color: rgb(0, 0, 0);
77     align-items: right;
78     border: 1px solid #000000;
79 }
80
81 a:last-of-type {
82     background: rgb(255, 255, 255);
83     color: #000000;
84     border: 1px solid #000000;
85 }

```

Código login.css para la parte de login.html

```

1  body {
2      font-family: Arial, sans-serif;
3      background-color: #ffffff;
4  }
5
6  .container {
7      width: 400px;
8      margin: 50px auto;
9      padding: 15px;
10     background-color: rgb(255, 255, 255);
11     border: 5px solid #ccc;
12     text-align: center;
13 }
14
15 h1 {
16     font-size: 28px;
17     color: #ffffff;
18     margin-bottom: 24px;
19 }
20
21
22 input {
23     width: 50%;
24     padding: 5px;
25     margin: 5px 0;
26 }
27
28 input[type="submit"] {
29     cursor: pointer;
30 }
31
32 img {
33     width: 100%;
34     max-width: 1600px;
35     height: 200px;
36     object-fit: cover;
37     border-radius: 8px;
38     margin-bottom: 10px;
39 }

```


Codigo register.css

```

1  body {
2      font-family: Arial, sans-serif;
3      background-color: #ffffff;
4  }
5
6  .container {
7      width: 400px;
8      margin: 50px auto;
9      padding: 30px;
10     background-color: white;
11     border: 5px solid #ccc;
12     text-align: center;
13 }
14
15
16 input {
17     width: 70%;
18     padding: 5px;
19     margin: 5px 0;
20 }
21
22 input[type="submit"] {
23     cursor: pointer;
24 }
25
26
27 img {
28     width: 100%;
29     max-width: 1600px;
30     height: 200px;
31     object-fit: cover;
32     border-radius: 8px;
33     margin-bottom: 10px;
34 }

```

Base de datos

Backup creado

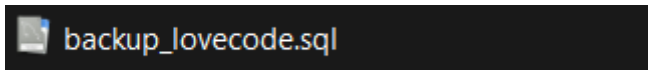


Tabla entidad relación

